

Design and Analysis of Algorithms

- **Course Code:** BCSE204L
- **Course Type:** Theory (ETH)
- **Slot:** A1+TA1 & & A2+TA2
- **Class ID:** VL2023240500901
VL2023240500902

A1+TA1

Day	Start	End
Monday	08:00	08:50
Wednesday	09:00	09:50
Friday	10:00	10:50

A2+TA2

Day	Start	End
Monday	14:00	14:50
Wednesday	15:00	15:50
Friday	16:00	16:50

Syllabus- Module 2

Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
----------	---	----------

Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP-

Backtracking: N-Queens problem, Subset Sum, Graph Coloring-

Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem

Branch and Bound (BB)

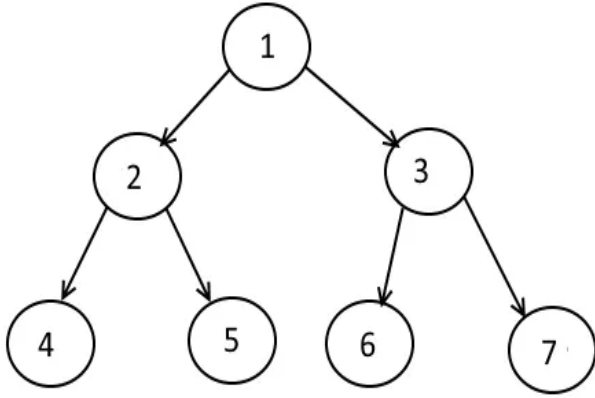
- Branch and Bound (BB) is indeed a problem-solving strategy commonly used for optimization problems.
 - Divides the problem into subproblems and assigns a bound to each subproblem, eliminating those that cannot lead to an optimal solution.
 - It operates by exploring the state space tree of a given problem, pruning branches that are not promising for finding the optimal solution. While similar to backtracking in terms of traversing the state space tree, BB differs in its approach and application.

Branch and Bound (BB)

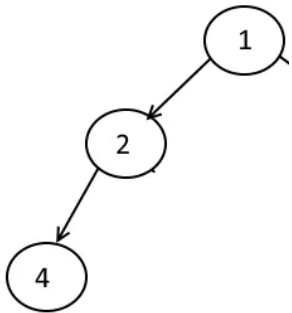
Here are some key points about Branch and Bound:

- **Optimization Problems:** BB is particularly useful for solving optimization problems, where the goal is to find the best solution among a set of feasible solutions. These problems often involve minimizing or maximizing an objective function subject to certain constraints.
- **Minimization vs. Maximization:** BB is typically applied to minimization problems. However, it is possible to adapt maximization problems into minimization ones. For instance, converting a maximization problem into a minimization one by taking the negative of the objective function.
- **Graph Traversal:** BB traverses the state space tree using a **breadth-first search** (BFS) approach rather than depth-first search (DFS) used by backtracking algorithms. This allows BB to systematically explore the entire state space in an organized manner.
- **Types of BB:**
 - **LIFO BB:** Implemented using a stack data structure, LIFO (Last-In-First-Out) BB explores nodes in a depth-first manner.
 - **FIFO BB:** Implemented using a queue data structure, FIFO (First-In-First-Out) BB explores nodes in a breadth-first manner.
 - **LC BB:** Implemented using a least-cost strategy, LC (Least Cost) BB prioritizes exploring nodes with the lowest estimated cost or bound.

Back Tracking approach (DFS)



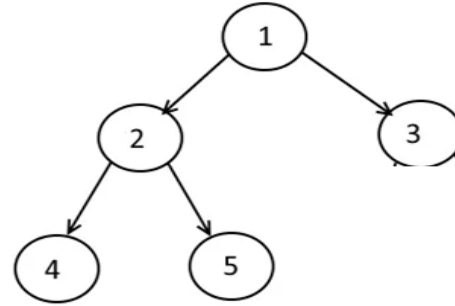
In backtracking, bounding function are used to help avoid the generation of subtrees that do not contain an answer node.



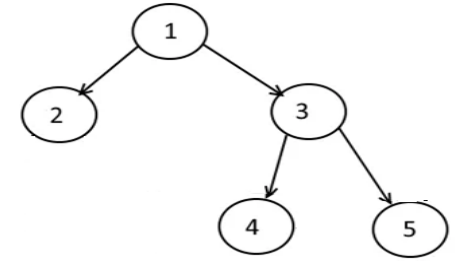
Branch and Bound (BB)

BB follows BFS (Breadth First search).

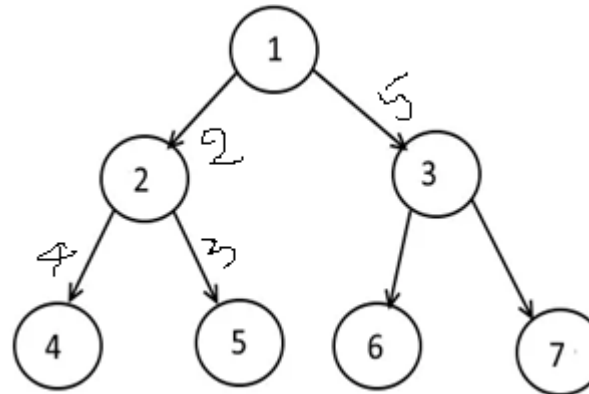
In BFS there is a possible to perform calculations in two different ways



FIFO BB: Here first node is expended, then next. That is First in First Out, implemented using Queue.



LIFO BB: Here last node is expended, then previous one. That is Last in First Out, implemented using Stack.



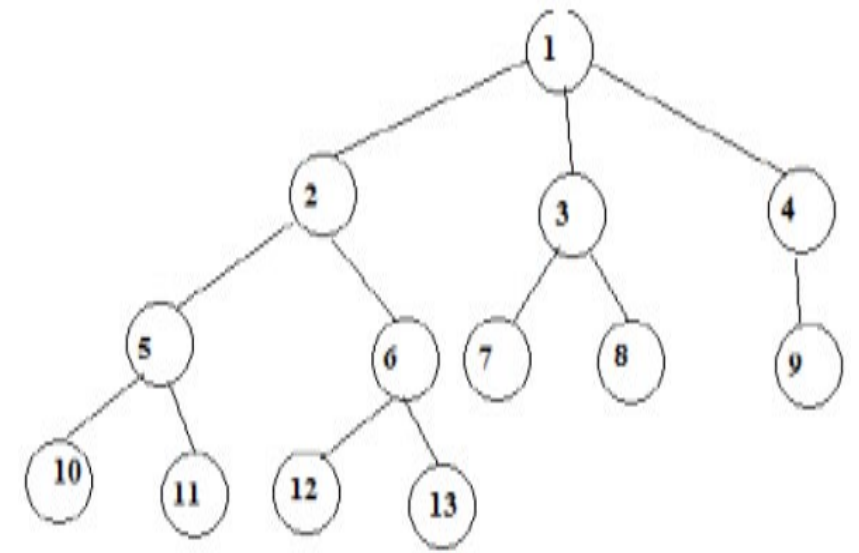
LC BB: Instead of following the queue or stack, a specific format is followed, that is least cost.

- **Active node** (Live node) is a node that has been generated but whose children have not yet been generated.
- **E-node** is an active node whose children are currently being explored.
- **Dead node** is a generated node that should not be expanded or explored further. All the children of a dead node have already been expanded

Branch-and-bound refers to all state space search methods in which **all children of an E-node** are generated **before** **any other live node can become the E-node.**

FIFO Branch and bound

- FIFO Branch and Bound is a BFS.
- In FIFO Branch and Bound , children of E-Node (or Live nodes) are inserted in a **queue**.
- Implementation of list of live nodes as a queue
 - **Least()** Removes the head of the Queue
 - **Add()** Adds the node to the end of the Queue



1

2 3 4

3 4 5 6

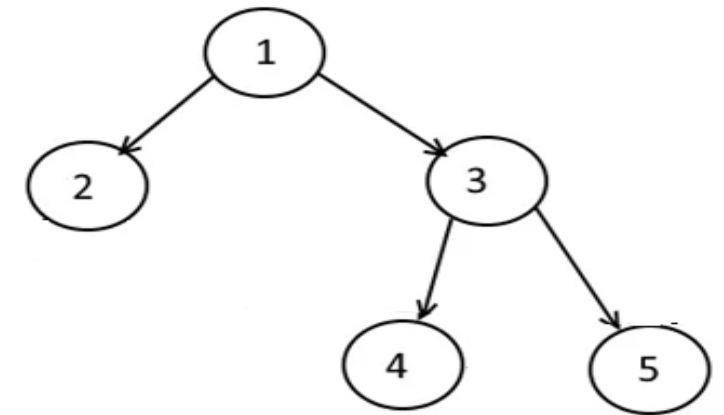
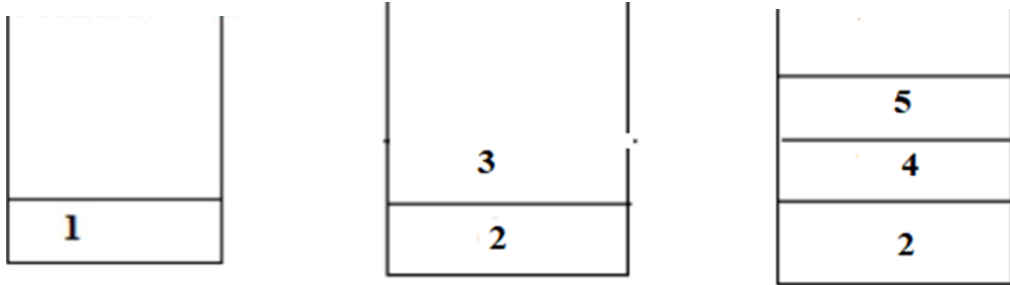
4 5 6

5 6

6

LIFO Branch and bound

- LIFO Branch and Bound is a **D-search** (or DFS).
- In LIFO Branch and Bound children of E-node (live nodes) are inserted in a stack
- Implementation of List of live nodes as a stack
 - Least() Removes the top of the stack
 - ADD() Adds the node to the top of the stack



Note: In both LIFO and FIFO BB's the selection of route or path for a required node is subsequently expended one after other nodes in BFS format.

That is, the selection rule for the **next E-node in rigid and blind**. The selection rule for the next E-node does not give any preference to a node that has a very good chance of getting the search to an answer node quickly.

In LC BB, considered the least cost node and expended them by avoiding other nodes.

The search for an answer node can be speeded by using an “intelligent” ranking function $c()$ for live nodes.

The next E-node is selected on the basis of this ranking function. The node x is assigned a rank using:

$$c(x) = f(h(x)) + g(x)$$

where,

$c(x)$ is the cost of x .

$h(x)$ is the cost of reaching x from the root and $f(.)$ is any non-decreasing function.

$g(x)$ is an estimate of the additional effort needed to reach an answer node from x .

Branch and Bound (BB) @ applications

- Job Selection problem
- Travelling sales person problem
- 0/1 knapsack problem

Parameter	Backtracking	Branch and Bound
Approach	Backtracking is used to find all possible solutions available to a problem. When it realises that it has made a bad choice, it undoes the last choice by backing it up. It searches the state space tree until it has found a solution for the problem.	Branch-and-Bound is used to solve optimisation problems. When it realises that it already has a better optimal solution that the pre-solution leads to, it abandons that pre-solution. It completely searches the state space tree to get optimal solution.
Traversal	Backtracking traverses the state space tree by <u>DFS(Depth First Search)</u> manner.	Branch-and-Bound traverse the tree in any manner, <u>DFS</u> or <u>BFS</u> .
Function	Backtracking involves feasibility function.	Branch-and-Bound involves a bounding function.
Problems	Backtracking is used for solving Decision Problem.	Branch-and-Bound is used for solving Optimisation Problem.
Searching	In backtracking, the state space tree is searched until the solution is obtained.	In Branch-and-Bound as the optimum solution may be present any where in the state space tree, so the tree need to be searched completely.
Efficiency	Backtracking is more efficient.	Branch-and-Bound is less efficient.
Applications	Useful in solving <u>N-Queen Problem</u> , <u>Sum of subset</u> .	Useful in solving <u>Knapsack Problem</u> , <u>Travelling Salesman Problem</u> .
Solve	Backtracking can solve almost any problem. (chess, sudoku, etc).	Branch-and-Bound can not solve almost any problem.

Job Selection problem

- **Job Selection problem** also known as the Job Sequencing problem. In this problem, there are n jobs, each with a deadline and profit associated with it. The objective is to select a subset of jobs to perform within their deadlines, maximizing the total profit while ensuring that no job is executed after its deadline.
- **Job Assignment Problem**, there are N workers and N jobs. Each worker can be assigned to perform any job, incurring a certain cost that may vary depending on the work-job assignment. The goal is to perform all jobs by assigning exactly one worker to each job and exactly one job to each worker in such a way that the total cost of the assignment is minimized.

Any

Question



PresenterMedia

Thank You!

**FOR YOUR
ATTENTION**

